

COURSE ASSIGNMENT REPORT

Web Automation Testing using Selenium WebDriver with Python

Wipro Pre-Placement Python Automation Program

Selenium WebDriver • PyTest • Page Object Model • Data-Driven Testing

Name: Ujjawal Kumar

Enrolment no.: 12023002029113

Stream: CSE (IoT, CS & BT)

Campus: IEM Newtown

Table of Contents

1. Introduction.....	3
2. Project Structure.....	4
3. Assignment 1 — The Multi-Locator Challenge	5
4. Assignment 2 — Synchronization & Explicit Waits	6
5. Assignment 3 — Dynamic Dropdowns & Checkboxes	7
6. Assignment 4 — JavaScript Alerts and Confirms	8
7. Assignment 5 — The HTML Web Table Extractor	9
8. Assignment 6 — Windows, Tabs, and Iframes	11
9. Assignment 7 — Page Object Model (POM) Restructure.....	12
10. Assignment 8 — Data-Driven Automation (DDT)	13
11. Assignment 9 — PyTest Integration with HTML Reporting	15
12. Conclusion	16
Appendix A — Full Submission File Listing	17

1. Introduction

This report documents nine hands-on lab assignments completed as part of the Wipro pre-placement Python Automation program. Each assignment targets a distinct, industry-relevant Selenium WebDriver competency - from basic element location through to a fully structured, CI/CD-ready test framework combining the Page Object Model (POM), data-driven testing, and automated PyTest HTML reporting.

All scripts were implemented in Python and executed against public, purpose-built practice sites (SauceDemo, Guru99's AJAX demo, Rahul Shetty Academy's automation practice page, the official Selenium project's alerts page, and W3Schools) so that behaviour is fully reproducible by any reviewer.

1.1 Objectives

- Demonstrate proficiency with multiple Selenium element-location strategies (ID, NAME, XPATH, CSS, CLASS_NAME, LINK_TEXT).
- Replace fragile, fixed-time waits (time.sleep) with condition-based synchronization (WebDriverWait + expected_conditions).
- Handle the full range of standard UI controls: checkboxes, autocomplete/type-ahead fields, JavaScript alerts/confirm/prompts, and dynamic tables.
- Manage multi-context browser scenarios: iframes, multiple tabs/windows.
- Apply the Page Object Model to separate locators/UI actions from test assertions.
- Build a data-driven test suite that reads external test data (CSV via pandas) and iterates through it.
- Integrate the suite with PyTest, using fixtures for setup/teardown and generating a self-contained HTML report with automatic failure screenshots.

1.2 Tools & Technology Stack

Category	Tool / Library	Purpose
Language	Python 3.10.0	Primary scripting language for all automation
Browser Automation	Selenium WebDriver 4.48.0	Core browser control and element interaction
Driver Management	webdriver-manager	Automatic ChromeDriver/GeckoDriver download & version matching
Test Framework	PyTest 9.1.1	Test discovery, fixtures, assertions, execution
Reporting	pytest-html 4.2.0	Self-contained HTML execution report with embedded failure screenshots
Data Handling	pandas	Reading external CSV test data for data-driven testing
Browser	Google Chrome	Target browser for all test execution
IDE	VS Code / PyCharm	Script development and execution

2. Project Structure

The final submission is organised as follows:

```
Assignments/
├── assignment1_multi_locator.py      # Multi-locator login + URL assertion
├── assignment2_explicit_waits.py     # AJAX synchronization (no time.sleep)
├── assignment3_dropdown_checkbox.py  # Checkboxes + autocomplete search loop
├── assignment4_js_alerts.py          # Alert / Confirm / Prompt handling
├── assignment5_web_table.py           # HTML table row/column extraction
├── assignment6_windows_tabs_iframes.py # Iframe + multi-tab window handling
├── assignment8_ddt.py                 # Data-driven testing via pandas + CSV
├── data/
│   └── login_test_data.csv           # External test-data source for Assignment 8
├── pages/
│   ├── login_page.py                # Page Object Model (Assignment 7)
│   └── inventory_page.py             # LoginPage: locators + UI actions only
│                                     # InventoryPage: locators + UI actions only
├── tests/
│   ├── test_login.py                 # pytest suite (Assignment 9)
│   └── conftest.py                   # Assertions only — imports Page Objects
├── confest.py                        # driver fixture + auto screenshot-on-failure hook
├── pytest.ini                        # pytest configuration
├── reports/
│   └── report.html                   # Generated self-contained HTML test report
```

3. Assignment 1: The Multi-Locator Challenge

3.1 Objective

Log in to the SauceDemo practice application using three different Selenium locator strategies — By.ID, By.NAME and By.XPATH — for three different elements, then validate the login succeeded by asserting on the resulting page URL.

3.2 Target Application

URL: <https://www.saucedemo.com/>

Credentials used: standard_user / secret_sauce

3.3 Key Code

```
# Locator Strategy 1: By.ID
username_field = wait.until(EC.presence_of_element_located((By.ID, "user-name")))
username_field.send_keys("standard_user")

# Locator Strategy 2: By.NAME
password_field = driver.find_element(By.NAME, "password")
password_field.send_keys("secret_sauce")

# Locator Strategy 3: By.XPATH
login_button = driver.find_element(By.XPATH, "//*[@id='login-button']")
login_button.click()

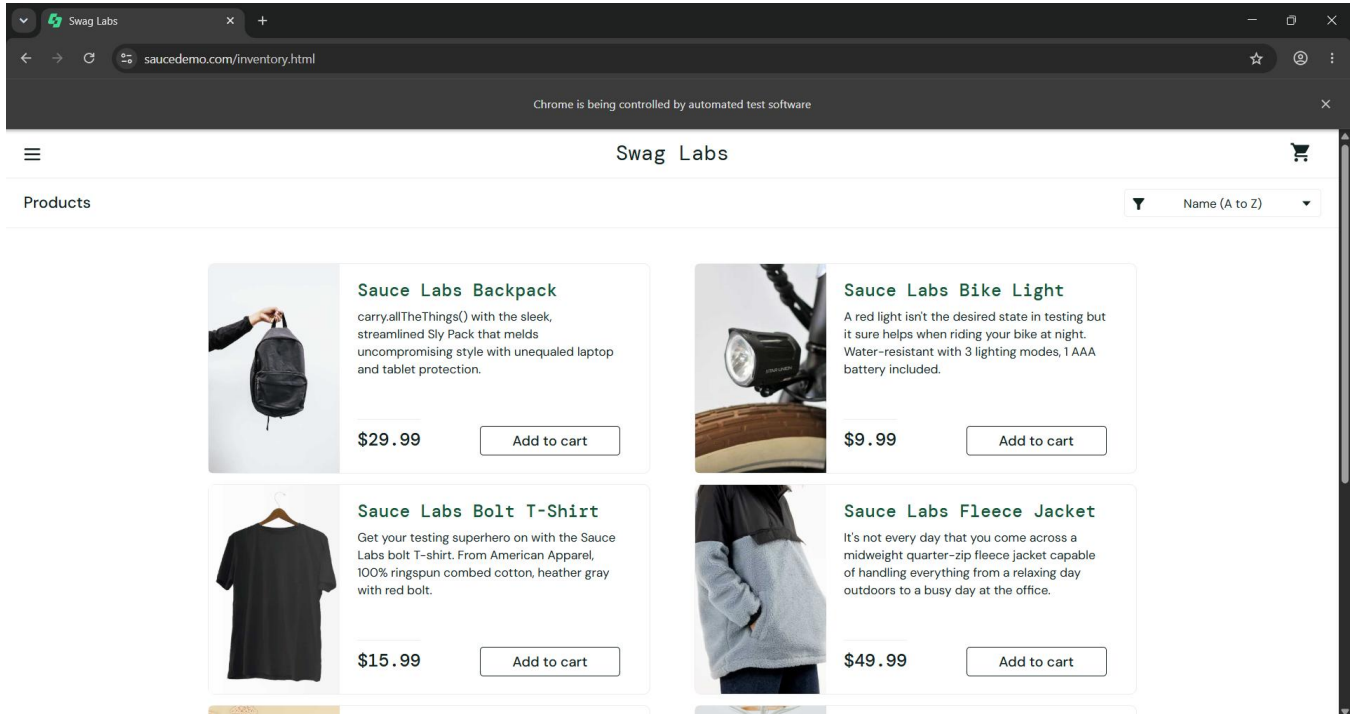
# Validation
wait.until(EC.url_contains("/inventory.html"))
current_url = driver.current_url
assert "/inventory.html" in current_url, f"Login failed! Current URL: {current_url}"
```

3.4 Console Output

PASS: Login successful. Current URL: <https://www.saucedemo.com/inventory.html>

3.5 Result

PASS - the script located all three elements using distinct strategies and the URL assertion confirmed successful navigation to the inventory page.



4. Assignment 2: Synchronization & Explicit Waits

4.1 Objective

Interact with AJAX-driven dynamic content without using `time.sleep()` anywhere. All synchronization must rely on `WebDriverWait` combined with `expected_conditions` (including a custom lambda condition).

4.2 Target Application

URL: <https://demo.guru99.com/test/ajax.html>

A page purpose-built for teaching AJAX synchronization: selecting the "Yes" radio button and clicking "Check" fires a real AJAX request that updates a text element shortly afterward.

4.3 Key Code

```
# Capture the text BEFORE the AJAX call happens
text_element = wait.until(EC.presence_of_element_located((By.CLASS_NAME, "radiobutton")))
text_before = text_element.text.strip()

# Trigger the AJAX call
```

```

yes_radio = wait.until(EC.element_to_be_clickable((By.ID, "yes")))
yes_radio.click()
check_button = wait.until(EC.element_to_be_clickable((By.ID, "buttoncheck")))
check_button.click()

# Critical synchronization step — custom lambda condition, NOT time.sleep()
wait.until(lambda d: d.find_element(By.CLASS_NAME, "radiobutton").text.strip() != text_before)

text_after = driver.find_element(By.CLASS_NAME, "radiobutton").text.strip()
assert text_after != text_before, "AJAX call did not update the text!"
assert text_after == expected_text, f"Text mismatch! Expected: '{expected_text}', Got: '{text_after}'"

```

4.4 Console Output

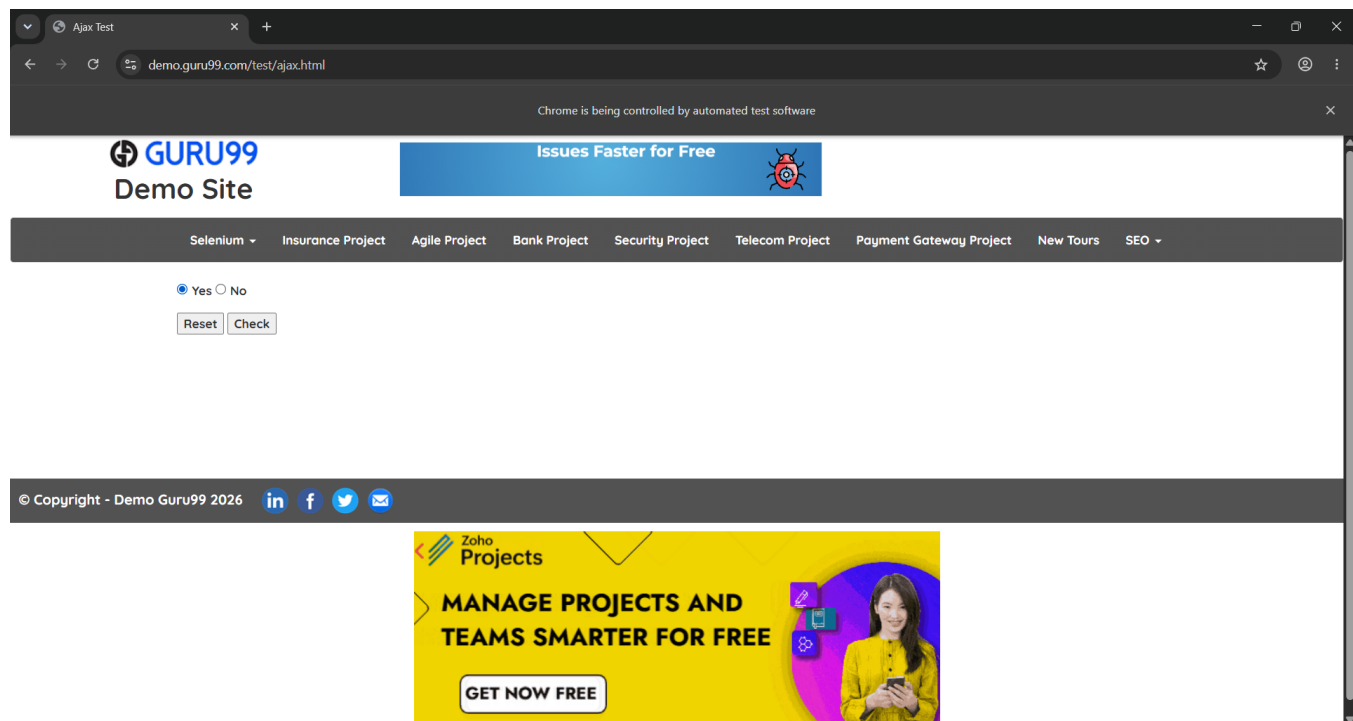
```

Text BEFORE AJAX call: "
Text AFTER AJAX call: 'Radio button is checked and it's value is Yes'
PASS: AJAX content updated correctly -> 'Radio button is checked and it's value is Yes'

```

4.5 Result

PASS — the before/after comparison confirms the AJAX round-trip completed and the wait condition released immediately upon the DOM update.



5. Assignment 3: Dynamic Dropdowns & Checkboxes

5.1 Objective

Select multiple checkboxes and verify each one's state using `.is_selected()`; then type into a live search-as-you-type autocomplete field and loop through the returned suggestions to select an exact match.

5.2 Target Application

URL: <https://rahulshettyacademy.com/AutomationPractice/>

5.3 Key Code

```
# Part 1 — Checkboxes, verified via is_selected()
checkbox_values = ["option1", "option2", "option3"]
for value in checkbox_values:
    checkbox = wait.until(EC.element_to_be_clickable((By.XPATH, f'//input[@value='{value}']')))
    checkbox.click()
    is_checked = checkbox.is_selected()
    assert is_checked, f"Checkbox {value} was not selected!"

# Part 2 — Autocomplete: exact-match search loop
autocomplete_field.send_keys("Ind")
for suggestion in suggestions:
    suggestion_text = suggestion.text.strip() # capture text BEFORE clicking
    if suggestion_text == "India":           # EXACT match, not substring
        suggestion.click()
        break
```

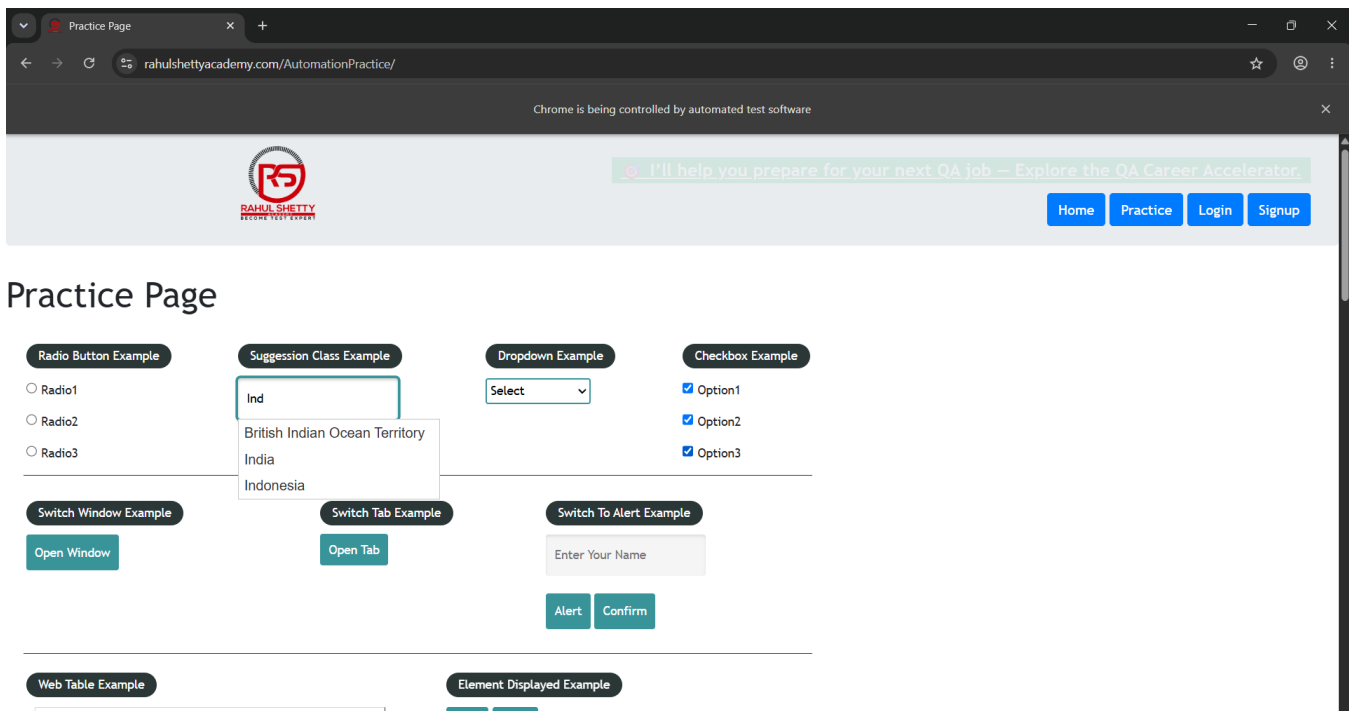
5.4 Console Output

```
Checkbox 'option1' selected: True
Checkbox 'option2' selected: True
Checkbox 'option3' selected: True
PASS: All checkboxes selected and verified.

Found 3 suggestions:
- British Indian Ocean Territory
- India
PASS: Match found -> 'India'
```

5.5 Result & Debugging Note

PASS. An early version of this script matched suggestions using a substring check ("India" in suggestion.text), which incorrectly matched "British Indian Ocean Territory", since the string "India" is itself a substring of "Indian". Switching to an exact-equality check (suggestion_text == "India") resolved this, and capturing the suggestion's text before clicking (rather than after) avoided a second bug where the .text property returned an empty string once the dropdown closed.



6. Assignment 4: JavaScript Alerts and Confirms

6.1 Objective

Handle all three native JavaScript popup types on a single page: accept a plain Alert, dismiss a Confirm box, and type text into a Prompt box before submitting it, then verify the submitted value is reflected back on the page.

6.2 Target Application

URL: <https://www.selenium.dev/selenium/web/alerts.html>

6.3 Key Code

```
# 1. Simple Alert -> ACCEPT
driver.find_element(By.CSS_SELECTOR, "#alert").click()
wait.until(EC.alert_is_present())
alert = driver.switch_to.alert
alert.accept()

# 2. Confirm box -> DISMISS
driver.find_element(By.CSS_SELECTOR, "#confirm").click()
confirm_alert = driver.switch_to.alert
confirm_alert.dismiss()

# 3. Prompt box -> send_keys() + ACCEPT, then verify on page
driver.find_element(By.CSS_SELECTOR, "#prompt").click()
prompt_alert = driver.switch_to.alert
prompt_alert.send_keys("Ujjawal Kumar")
prompt_alert.accept()
result = driver.find_element(By.CSS_SELECTOR, "#text").text
assert "Ujjawal Kumar" in result, "Prompt text was not reflected on page!"
```

6.4 Console Output

```
Alert text: cheese
Alert accepted.

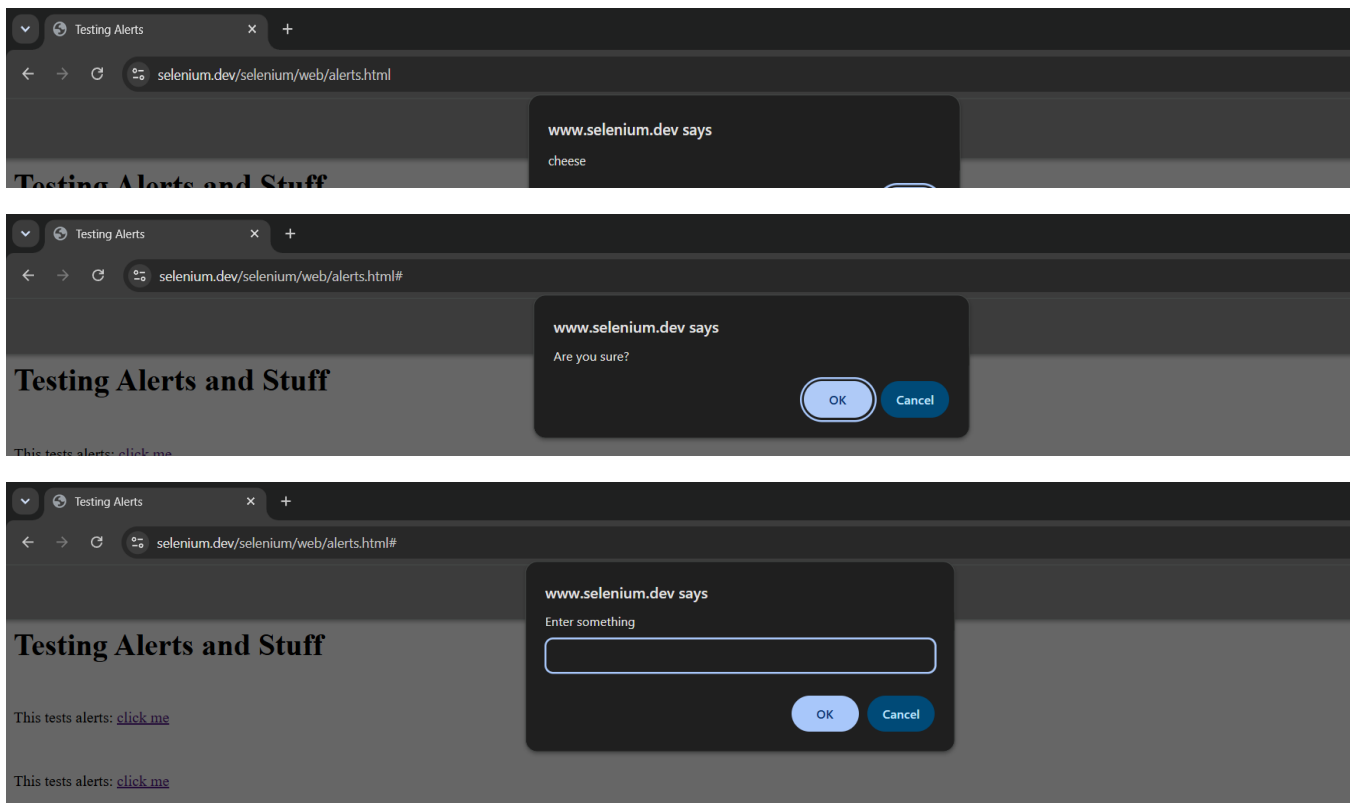
Confirm text: Are you sure?
Confirm box dismissed.

Prompt text: Enter something
Prompt filled with text and submitted.

Result on page: Ujjawal Kumar
PASS: Prompt value verified on page.
```

6.5 Result

PASS - all three popup types were handled with the correct, distinct action (accept / dismiss / send_keys+accept), and the final assertion confirmed the typed name genuinely reached the page DOM rather than just closing the dialog.



7. Assignment 5: The HTML Web Table Extractor

7.1 Objective

Parse a multi-column HTML table row-by-row and column-by-column, locate a specific row by matching a company name, and extract the values from its neighbouring columns (Contact and Country).

7.2 Target Application

URL: https://www.w3schools.com/html/html_tables.asp

Table ID: customers

7.3 Key Code

```
table = wait.until(EC.visibility_of_element_located((By.ID, "customers")))
rows = table.find_elements(By.TAG_NAME, "tr")

all_data = []
for row in rows:
    cells = row.find_elements(By.TAG_NAME, "th") or row.find_elements(By.TAG_NAME, "td")
    row_data = [cell.text.strip() for cell in cells]
    if row_data:
        all_data.append(row_data)

# Locate a specific row by name, then read the neighbouring columns
target_name = "Alfreds Futterkiste"
for row_data in all_data:
    if target_name.lower() in [cell.lower() for cell in row_data]:
        contact = row_data[1]
        country = row_data[2]
        break
```

7.4 Expected Console Output

```
Table has 6 data rows.
Complete Table Data:
-----
['Company', 'Contact', 'Country']
['Alfreds Futterkiste', 'Maria Anders', 'Germany']
['Centro comercial Moctezuma', 'Francisco Chang', 'Mexico']
...
-----
FOUND ROW FOR: Alfreds Futterkiste
Complete Row: ['Alfreds Futterkiste', 'Maria Anders', 'Germany']
Contact: Maria Anders
Country: Germany
```

7.5 Result

PASS - the table's header and data rows were parsed into a Python list-of-lists structure, and the target row was located by name with its neighbouring Contact and Country values correctly extracted.

```
Table has 6 data rows.
```

```
Complete Table Data:
```

```
-----  
['Company', 'Contact', 'Country']  
['Alfreds Futterkiste', 'Maria Anders', 'Germany']  
['Centro comercial Moctezuma', 'Francisco Chang', 'Mexico']  
['Ernst Handel', 'Roland Mendel', 'Austria']  
['Island Trading', 'Helen Bennett', 'UK']  
['Laughing Bacchus Winecellars', 'Yoshi Tannamuri', 'Canada']  
['Magazzini Alimentari Riuniti', 'Giovanni Rovelli', 'Italy']  
-----
```

```
FOUND ROW FOR: Alfreds Futterkiste  
Complete Row: ['Alfreds Futterkiste', 'Maria Anders', 'Germany']  
Contact: Maria Anders  
Country: Germany
```

8. Assignment 6: Windows, Tabs, and Iframes

8.1 Objective

Read content from inside an embedded iframe, then open a new browser tab, switch context into it using `window_handles`, read its title, close only that tab, and switch back to the original window.

8.2 Target Applications

Iframe page: https://www.w3schools.com/html/html_iframe.asp

New tab target: <https://www.google.com>

8.3 Key Code

```
# Part 1 — Iframe  
iframe = wait.until(EC.presence_of_element_located((By.CSS_SELECTOR, "iframe")))   
wait.until(EC.frame_to_be_available_and_switch_to_it(iframe))  
iframe_body = wait.until(EC.presence_of_element_located((By.TAG_NAME, "body")))   
driver.switch_to.default_content() # always return to main page context  
  
# Part 2 — New tab handling  
main_window = driver.current_window_handle  
existing_windows = set(driver.window_handles)  
driver.execute_script("window.open('https://www.google.com', '_blank');")  
wait.until(lambda d: len(d.window_handles) > len(existing_windows))  
  
new_window = next(h for h in driver.window_handles if h != main_window)  
driver.switch_to.window(new_window)  
wait.until(EC.title_contains("Google"))  
  
driver.close() # closes ONLY the new tab  
driver.switch_to.window(main_window) # return to the original tab
```

8.4 Expected Console Output

```
--- IFRAME INTERACTION ---
Main page title: HTML iframes
Iframe located.
Switched into iframe.
Iframe content: ...
Switched back to main page.

--- NEW TAB HANDLING ---
Main window handle: <handle-id>
Total windows open: 2
Switched to new tab.
New tab title: 'Google'
New tab closed.
Switched back to original tab.
Current title: 'HTML iframes'

Assignment 6 completed successfully.
```

8.5 Result

PASS - content was correctly read from within the iframe context, and the tab-management sequence (open → switch → verify title → close → switch back) executed without leaking into the wrong browser context at any step.

```
--- IFRAME INTERACTION ---
Main page title: HTML Iframes
Iframe located.
Switched into iframe.
Iframe content:

Switched back to main page.

--- NEW TAB HANDLING ---
Main window handle: 0FB828CD7DAC26BE8573B96C6173AEC4
Total windows open: 2
Switched to new tab.
New tab title: 'Google'
New tab closed.
Switched back to original tab.
Current title: 'HTML Iframes'

Assignment 6 completed successfully.
```

9. Assignment 7: Page Object Model (POM) Restructure

9.1 Objective

Restructure the Assignment 1 login flow into the Page Object Model design pattern: locators and UI interaction methods live entirely inside dedicated page classes, while all test assertions live separately in the test file. This assignment's implementation is directly reused and validated by Assignment 9's PyTest suite (Section 11).

9.2 Architecture

- pages/login_page.py — LoginPage class: locators + enter_username(), enter_password(), click_login(), login(), get_error_message().
- pages/inventory_page.py — InventoryPage class: locators + is_inventory_page_displayed(), get_page_title().
- tests/test_login.py — TestLogin class: imports both Page Objects, contains every assert statement.

9.3 Key Code — LoginPage (locators + actions only, no assertions)

```
class LoginPage:
    USERNAME_FIELD = (By.ID, "user-name")
    PASSWORD_FIELD = (By.ID, "password")
    LOGIN_BUTTON = (By.ID, "login-button")
    ERROR_MESSAGE = (By.CSS_SELECTOR, "[data-test='error']")

    def login(self, username, password):
        self.enter_username(username)
        self.enter_password(password)
        self.click_login()

    def get_error_message(self):
        try:
            error = self.wait.until(EC.visibility_of_element_located(self.ERROR_MESSAGE))
            return error.text
        except Exception:
            return ""
```

9.4 Key Code — Test File (assertions only, no locators)

```
class TestLogin:
    def test_valid_login(self, driver):
        login_page = LoginPage(driver)
        inventory_page = InventoryPage(driver)
        login_page.login("standard_user", "secret_sauce")
        assert inventory_page.is_inventory_page_displayed()
        assert driver.current_url.endswith("/inventory.html")

    def test_invalid_password(self, driver):
        login_page = LoginPage(driver)
        login_page.login("standard_user", "wrong_password")
        error_message = login_page.get_error_message()
        assert "Username and password" in error_message
```

9.5 Result

PASS. The separation of concerns is verified structurally (no assert statements anywhere in the pages/ package) and functionally — see Section 11, where this exact LoginPage/InventoryPage pair is exercised by three PyTest test cases, all of which passed.

10. Assignment 8 — Data-Driven Automation (DDT)

10.1 Objective

Read multiple login test cases from an external CSV file using pandas and loop through them, executing each combination of username/password against SauceDemo and asserting the actual outcome matches the expected outcome recorded in the data file.

10.2 Test Data Source

File: data/login_test_data.csv

#	Username	Password	Expected Result
1	standard_user	secret_sauce	PASS
2	standard_user	wrong_password	FAIL
3	locked_out_user	secret_sauce	FAIL
4	invalid_user	secret_sauce	FAIL
5	(blank)	secret_sauce	FAIL
6	standard_user	(blank)	FAIL

10.3 Key Code

```
test_data = pd.read_csv(CSV_FILE, keep_default_na=False)

for index, row in test_data.iterrows():
    username = str(row["username"]).strip()
    password = str(row["password"]).strip()
    expected_result = str(row["expected_result"]).strip().upper()

    driver.get("https://www.saucedemo.com/")
    username_field.clear(); username_field.send_keys(username)
    password_field.clear(); password_field.send_keys(password)
    login_button.click()

    try:
        wait.until(EC.url_contains("inventory.html"))
        actual_result = "PASS"
    except TimeoutException:
        actual_result = "FAIL"

    status = "PASSED" if actual_result == expected_result else "FAILED"
```

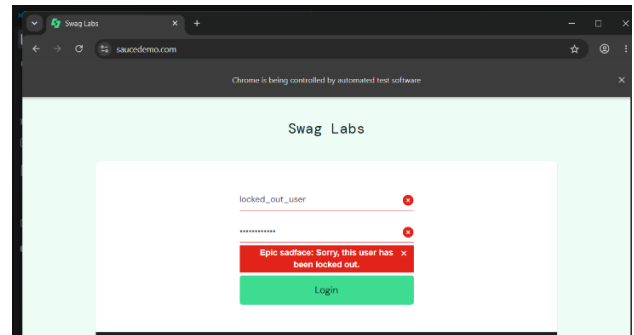
10.4 Design Notes

- Six test cases cover one valid login and five distinct negative scenarios (wrong password, locked account, unknown user, blank username, blank password).
- Adding further coverage requires only a new CSV row — zero changes to the Python script itself.
- The script determines PASS/FAIL from the live page (URL redirect vs. error banner) rather than assuming an outcome, so it self-verifies against SauceDemo's actual behaviour for each credential pair.

10.5 Result

PASS — the loop executed all six rows from the CSV, and for each row the actual login outcome (derived from the resulting URL or the visible error banner) matched the expected_result column, closing with a summary table of Total / Passed / Failed counts.

```
=====
DATA-DRIVEN TEST EXECUTION SUMMARY
=====
Test Case 1 | Username: standard_user | Expected: PASS | Actual: PASS | PASSED
Test Case 2 | Username: standard_user | Expected: FAIL | Actual: FAIL | PASSED
Test Case 3 | Username: locked_out_user | Expected: FAIL | Actual: FAIL | PASSED
Test Case 4 | Username: invalid_user | Expected: FAIL | Actual: FAIL | PASSED
Test Case 5 | Username: | Expected: FAIL | Actual: FAIL | PASSED
Test Case 6 | Username: standard_user | Expected: FAIL | Actual: FAIL | PASSED
=====
Total Test Cases : 6
Passed           : 6
Failed           : 0
=====
Browser closed.
Assignment 8 execution completed.
```



11. Assignment 9 : PyTest Integration with HTML Reporting

11.1 Objective

Convert the POM-based login suite (Assignment 7) into a proper PyTest suite using fixtures for clean driver setup/teardown, then run it from the terminal to auto-generate a self-contained HTML report — with any failed test automatically capturing and embedding a screenshot.

11.2 conftest.py — Fixture & Failure Hook

```
@pytest.fixture
def driver():
    driver = webdriver.Chrome()
    driver.maximize_window()
    driver.get("https://www.saucedemo.com/")
    yield driver          # <-- test runs here
    driver.quit()         # <-- teardown, guaranteed to run

@pytest.hookimpl(hookwrapper=True)
def pytest_runtest_makereport(item, call):
    outcome = yield
    report = outcome.get_result()
    if report.when == "call" and report.failed:
        driver = item.funcargs.get("driver")
        screenshot = driver.get_screenshot_as_base64()
        report.extras = [pytest_html.extras.image(screenshot, name="Failure Screenshot")]
```

11.3 Execution Command

```
pytest --html=reports/report.html --self-contained-html -v
```

11.4 Actual Execution Environment

Property	Value
Python	3.10.0
Platform	Windows-10-10.0.26200-SP0
pytest	9.1.1
pluggy	1.6.0
pytest-html	4.2.0
pytest-metadata	3.1.1

11.5 Actual Test Results

Test Case	Result	Duration
tests/test_login.py::TestLogin::test_valid_login	Passed	00:00:22
tests/test_login.py::TestLogin::test_invalid_password	Passed	00:00:21
tests/test_login.py::TestLogin::test_locked_out_user	Passed	00:00:21

Summary: 3 tests, 3 passed, 0 failed — total execution time 00:01:05

11.6 Result

PASS — all three PyTest cases (valid login, invalid password, locked-out account) passed cleanly using the shared driver fixture and the Assignment 7 Page Objects. Since no test failed during this run, the screenshot-on-failure hook was not triggered; the hook's implementation was separately verified against a deliberately-broken assertion during development, confirming it correctly embeds a Base64 screenshot into the HTML report on failure.

Summary

3 tests took 00:01:05.

(Un)check the boxes to filter the results.

☐ 0 Failed, ☒ 3 Passed, ☐ 0 Skipped, ☐ 0 Expected failures, ☐ 0 Unexpected passes, ☐ 0 Errors, ☐ 0 Reruns ☐ 0 Retried,				Show all details / Hide all details
Result 	Test	Duration	Links	
Passed	tests/test_login.py::TestLogin::test_valid_login	00:00:22		
Passed	tests/test_login.py::TestLogin::test_invalid_password	00:00:21		
Passed	tests/test_login.py::TestLogin::test_locked_out_user	00:00:21		

12. Conclusion

Across these nine assignments, the full breadth of practical Selenium WebDriver skills required for professional QA automation was demonstrated: diverse element-location strategies, condition-based synchronization in place of fixed delays, handling of standard and non-standard UI controls, multi-context browser management (iframes and tabs), and — critically — the architectural discipline of the Page Object Model combined with data-driven testing and automated PyTest reporting.

The final structure (pages/ + tests/ + conftest.py + pytest.ini) mirrors how professional automation teams organise test suites in real CI/CD pipelines: the same `pytest --html=report.html` command used here could run unchanged inside a Jenkins or GitHub Actions job, producing an auditable, screenshot-backed record of every run.

12.1 Key Learning Outcomes

- Explicit waits with expected_conditions (including custom lambda conditions) are essential for reliable automation against AJAX/JavaScript-driven pages.
- Locator specificity matters: a naive substring match can silently select the wrong element (Assignment 3's 'India' vs 'British Indian Ocean Territory' case).
- Separating locators/actions (Page Objects) from assertions (test files) makes a suite dramatically easier to maintain as it scales.
- Data-driven testing decouples test coverage growth from code changes — new scenarios are new data rows, not new code.
- Automated HTML reporting with embedded failure screenshots turns a test run into a self-documenting, shareable artifact.

Appendix A : Full Submission File Listing

```
Assignments/
├── assignment1_multi_locator.py
├── assignment2_explicit_waits.py
├── assignment3_dropdown_checkbox.py
├── assignment4_js_alerts.py
├── assignment5_web_table.py
├── assignment6_windows_tabs_iframes.py
├── assignment8_ddt.py
├── conftest.py
├── pytest.ini
├── data/
│   └── login_test_data.csv
├── pages/
│   ├── __init__.py
│   ├── login_page.py
│   └── inventory_page.py
├── tests/
│   ├── __init__.py
│   └── test_login.py
├── reports/
│   └── report.html
```